

Postman and PostgreSQL

What is Postman?

Postman is a tool used to test APIs. When we build a FastAPI backend, and create endpoints like:

- POST /login
- GET /users
- POST /projects

Postman allows you to send requests to these endpoints and see responses. A manual tester for your backend before the frontend is ready.

Why Do We Need Postman?

You made this API:

```
@app.post("/login")
```

```
def login():
```

How will you test it? Without frontend?

You use Postman to:

- Send request
- See response
- Check status code
- Debug errors

Postman vs swagger?

Swagger is a tool that comes with FastAPI and shows your API documentation. It lets you see all endpoints and test them quickly, which is good for small checks during development. Postman, on the other hand, is a professional API testing tool. It lets you organize requests in collections, use environment variables like `{{base_url}}`, handle tokens automatically, write test scripts, and share APIs with your team. So, while Swagger is useful for quick testing and documentation, Postman is much better for real projects, automation, and teamwork.

We use Postman instead of Swagger because:

Swagger = Documentation + Basic Testing

Postman = Professional API Testing + Automation + Environment Management

Swagger is like a small calculator. Postman is like a scientific calculator.

STEP-BY-STEP: How to Use Postman

STEP 1 — Send Basic Request

Your FastAPI running on: `http://127.0.0.1:8000`

Testing GET request

1. Open Postman
2. Click **New Request**
3. Select method: GET
4. Write URL: `http://127.0.0.1:8000/users`
5. Click **Send**

You will see: Status: 200 OK or JSON response

Sending POST Request

Example: Register user

In Postman:

1. Change method → POST
2. Go to **Body**
3. Select → raw
4. Select → JSON
5. Write:

```
{
```

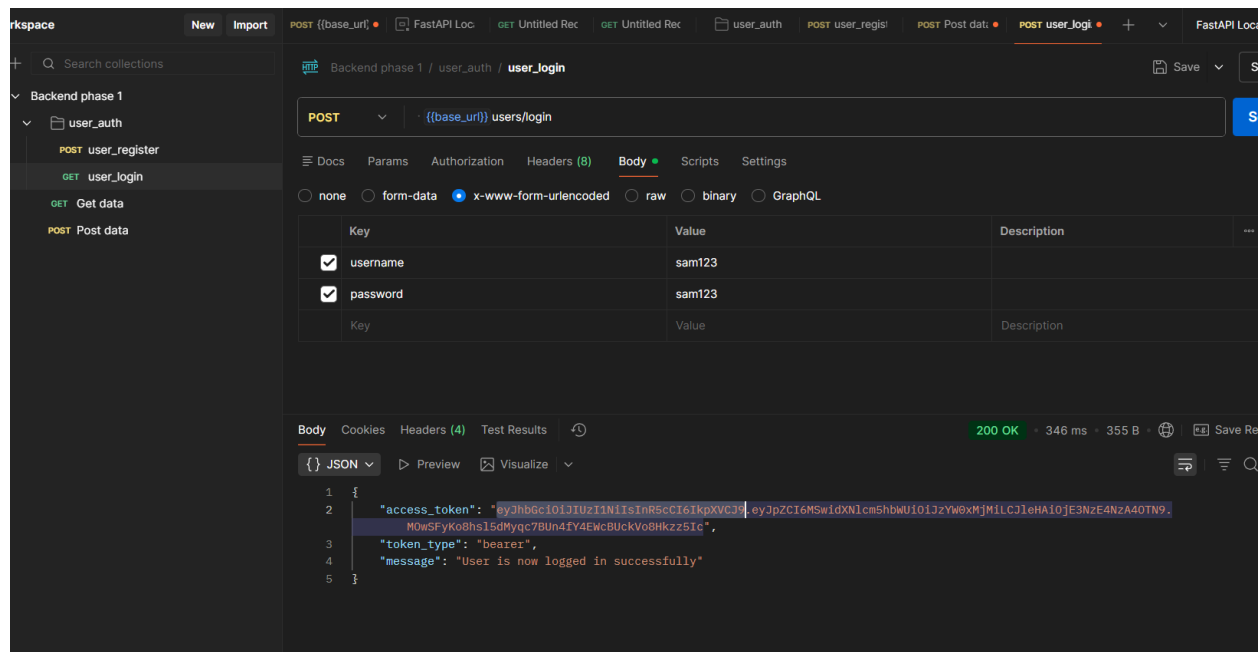
```
"username": "sameen",
```

"email": "sameen@gmail.com",

"password": "1234"

}

6. Click Send



Environment Handling:

You are building backend. Imagine:

Local: `http://127.0.0.1:8000`

Staging: `https://staging.api.com`

Production: `https://api.com`

In Swagger: You manually change URL.

In Postman: You create environments. `{{base_url}}`. Just switch environment. Done.

How to Create Environment:

Environment = Store variables like `base_url` and `token`.

Instead of writing:

`http://127.0.0.1:8000`

We write:

```
{{base_url}}
```

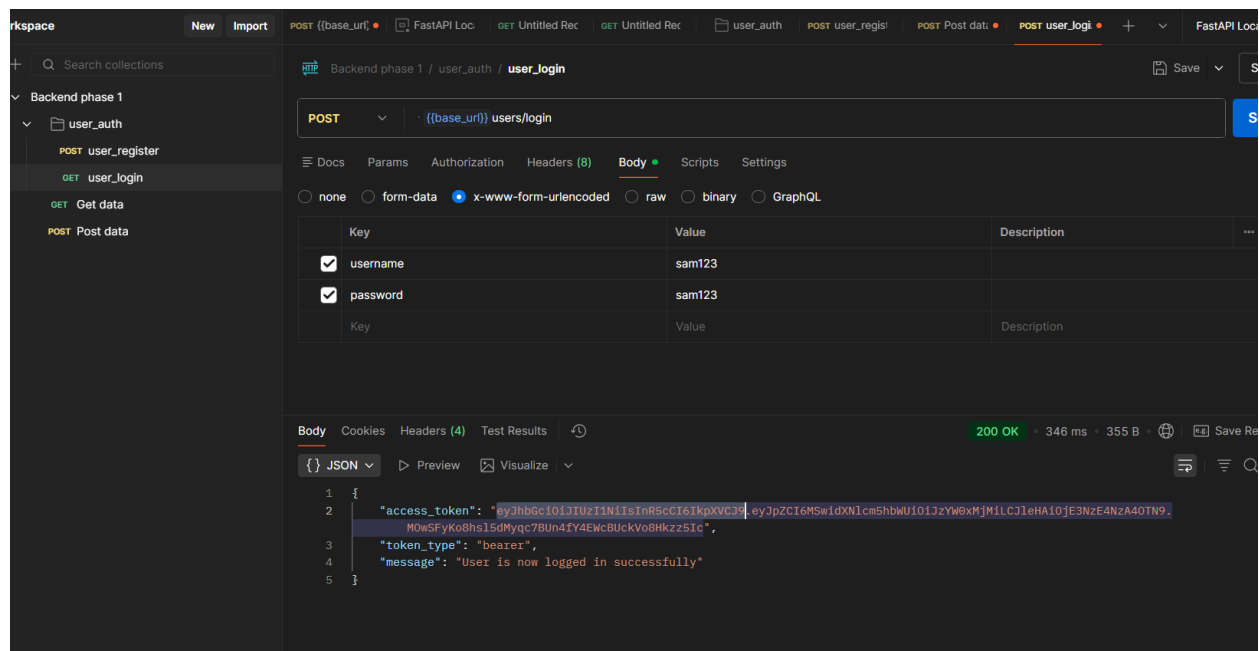
How to create environment:

1. Click top right → Environment → Create New
2. Name it:Fastapi local

Add variables:

Variable	Initial Value
base_url	http://127.0.0.1:8000
access_token	

Click Save.



Token Handling (JWT Case):

You are using JWT login.In Swagger:

You click "Authorize"

Paste token

In Postman:

You store token like this:{{access_token}}

Then in Headers:Authorization: Bearer {{access_token}}.you can even auto-save token from login response.

Example in Tests tab:

```
pm.environment.set("access_token", pm.response.json().access_token);
```

STEP to Add Authorization

After login:

- Copy token from response body of login request
- Go to other request
- Click Authorization
- Select Bearer Token
- Paste token
- Send

OR

Add manually in Headers:

Key: Authorization

Value: Bearer <your_token>

Save JWT Automatically

Now important professional step.

Go to **Tests tab/scripts** in login request.

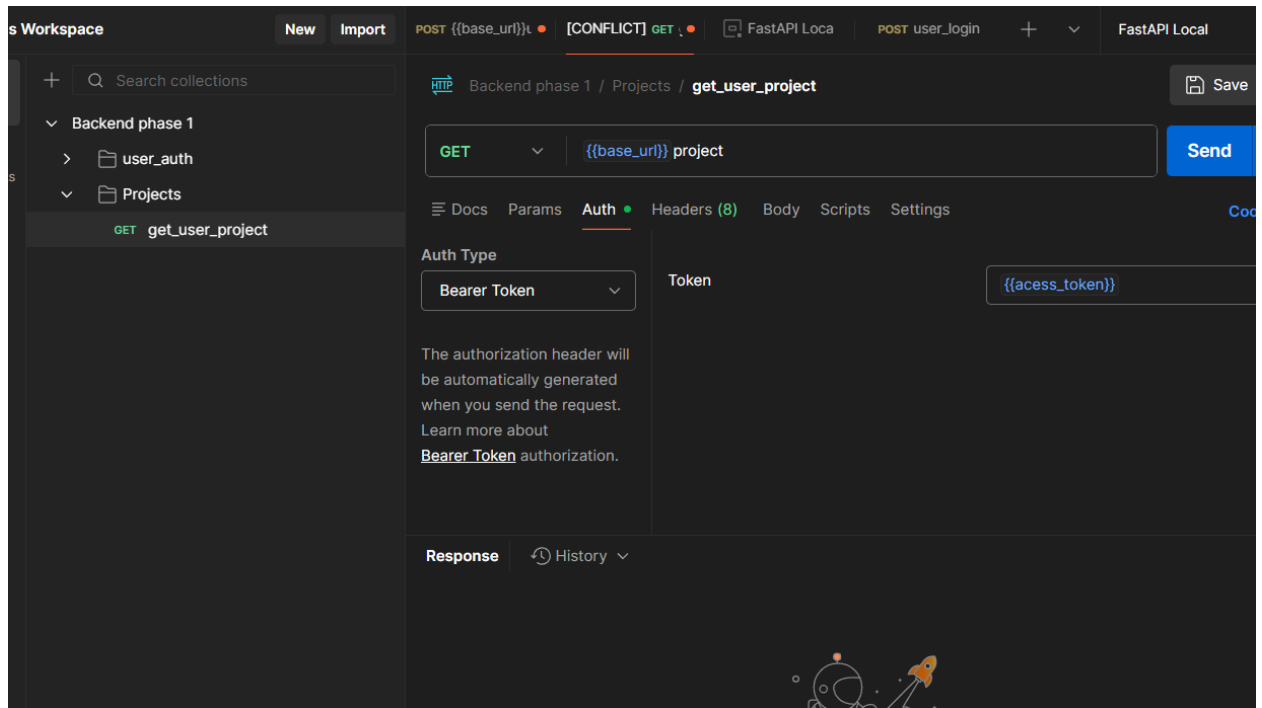
Write this:

```
let response = pm.response.json();  
pm.environment.set("access_token", response.access_token);
```

Click Save.

Now when you login → token automatically saved in environment variable.

You can check in Environment → access_token now has value.



Collections

Collections = folder of APIs

Why important?

- Organizes project
- Shareable
- Exportable
- Used in interviews

How to Create:

- Click Collections
- Click New Collection
- Name it: backend_phase1
- Save requests inside it

Benefits

- Used in almost every backend job
- Required for freelancing
- Required for testing JWT
- Required for OAuth2

PostgreSQL

PostgreSQL is a database.

Database = place where data is stored.

It stores data in tables.

Benefits of PostgreSQL

1. Open source (Free)
2. Very secure
3. Supports relationships
4. Handles large data
5. ACID compliant (Safe transactions)
6. Works perfectly with FastAPI + SQLAlchemy

Important PostgreSQL Concepts You Must Learn

Basic PostgreSQL Commands

Create Database

```
CREATE DATABASE mydb;
```

Create Table

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50),  
    email VARCHAR(100)  
);
```

Let's explain difficult terms:

- SERIAL → Auto increment number
- PRIMARY KEY → Unique id
- VARCHAR → Text with limit

Insert Data

```
INSERT INTO users (username, email)
```

```
VALUES ('mubeen', 'mubeen@gmail.com');
```

Select Data

```
SELECT * FROM users;
```

Update

```
UPDATE users
```

```
SET username = 'sam'
```

```
WHERE id = 1;
```

Delete

```
DELETE FROM users
```

```
WHERE id = 1;
```

Important PostgreSQL Functions

COUNT()

```
SELECT COUNT(*) FROM users;
```

Counts rows.

SUM()

```
SELECT SUM(price) FROM products;
```

Adds numbers.

AVG()

```
SELECT AVG(age) FROM users;
```

Average.

NOW()

```
SELECT NOW();
```

Gives current date and time.